

Automated Fruit Sorting System

Project Draft — Design & Implementation Guide (USB Webcam + Edge Impulse Edition)

1. Project Overview

This project builds an intelligent, configurable fruit sorting system using a Raspberry Pi as the central controller. A USB webcam feeds into an Edge Impulse machine learning model for visual fruit quality assessment, while a weight sensor checks whether the fruit meets the minimum weight threshold. A combined scoring logic merges both sensor outputs, and a servo motor routes each fruit into the correct basket (Good or Bad). The Edge Impulse model is trained on your laptop using a hybrid dataset from Kaggle and your own webcam images, then deployed onto the Raspberry Pi.

2. Hardware Components

Component	Purpose	Notes
Raspberry Pi 4 (recommended)	Central controller	Runs Edge Impulse model + logic
USB Webcam	Visual fruit inspection	Plug-and-play via USB port
Load Cell + HX711 Amplifier	Measure fruit weight	Compare against set threshold
Servo Motor	Conveyor sorting mechanism	Routes fruit to correct basket
Conveyor Belt	Transport fruit	Driven by motor
2x Baskets/Bins	Collect good/bad fruit	Labelled Good and Bad
Power Supply	Power all components	5V for Pi; external supply for servo
Laptop/PC	Edge Impulse training	Used for model training only

3. Laptop vs Raspberry Pi — Who Does What?

Your laptop handles all training and setup. The Raspberry Pi only runs the final deployed model.

Task	Where to Do It
Create Edge Impulse account	Laptop — edgeimpulse.com
Create a new project	Laptop — Edge Impulse dashboard
Download Kaggle dataset	Laptop — kaggle.com
Capture own webcam images (30-50)	Laptop or Pi webcam

Upload combined dataset to Edge Impulse	Laptop — Edge Impulse dashboard
Design the Impulse (model pipeline)	Laptop — Edge Impulse dashboard
Train the ML model	Laptop — Edge Impulse cloud (automatic)
Test and validate the model	Laptop — Edge Impulse dashboard
Download the trained model (.eim file)	Laptop — then copy to Pi
Install Edge Impulse Linux SDK	Raspberry Pi terminal
Run live inference with webcam	Raspberry Pi
Control servo motor & sorting logic	Raspberry Pi

4. Dataset Collection — Hybrid Approach (Kaggle + Own Images)

Capturing 150-200 images manually is time-consuming. Instead, we recommend a hybrid approach: use Kaggle datasets for the bulk of your training data, and add a small set of your own webcam images to adapt the model to your real environment. This saves significant time while maintaining good real-world accuracy.

Important — What is a Domain Gap?

Kaggle images are taken under professional lighting, high-end cameras, and clean backgrounds. Your USB webcam on the conveyor sees fruit very differently. This mismatch is called a domain gap and can reduce real-world accuracy even if your model scores well during training. Adding your own images bridges this gap.

Step 1 — Download a Kaggle Dataset

Go to kaggle.com on your laptop and search for fruit quality datasets. Recommended search terms:

- "fruit quality detection dataset"
- "fresh vs rotten fruit dataset"
- "fruit and vegetable disease healthy vs rotten"
- "fresh and stale fruit classification"

What to look for in a Kaggle dataset:

- At least 200+ images per class (good/fresh and bad/rotten)
- Images that match your fruit type (apple, mango, banana, etc.)
- Variety in angles, lighting, and backgrounds — not just studio shots
- Check the licence — most Kaggle datasets allow educational/personal use
- Download only the classes you need — good_fruit and bad_fruit

Step 2 — Capture Your Own Webcam Images (30-50)

This small batch is the most important part — it teaches the model your specific camera, lighting, and conveyor setup:

- Set up your USB webcam exactly as it will be positioned on the conveyor
- Use the same lighting you will use during actual sorting
- Capture 30-50 images of good fruit and 30-50 images of bad fruit
- Vary angles slightly — but keep the conveyor background consistent
- Even a small number of real-environment images greatly improves accuracy

Step 3 — Combine and Upload to Edge Impulse

- Organise your images into two folders: `good_fruit` and `bad_fruit`
- Mix Kaggle images and your own webcam images in both folders
- Recommended ratio: ~150-200 Kaggle images + 30-50 own images per class
- Upload to Edge Impulse dashboard via drag-and-drop or the Data Acquisition tab
- Label each image correctly during upload — `good_fruit` or `bad_fruit`
- Edge Impulse will automatically split: 80% training, 20% testing

Pro Tip: After your first deployment, run the system on 20-30 real fruits and check the results. If accuracy is low, add more of your own webcam images and retrain — this is the fastest way to improve real-world performance.

5. Edge Impulse — Model Training on Laptop

Step 1 — Create Your Account & Project

- Open your laptop browser and go to edgeimpulse.com
- Click 'Sign Up' and create a free account — verify your email
- Click 'Create New Project' → select 'Image Classification'
- Name your two classes: `'good_fruit'` and `'bad_fruit'`

Step 2 — Design the Impulse

- Go to 'Create Impulse' in your project dashboard
- Input block: set image size to 96x96 pixels (good balance of speed and accuracy)
- Processing block: select 'Image' → choose RGB colour mode
- Learning block: select 'Transfer Learning (Images)'
- Recommended model: MobileNetV2 96x96 — fast and accurate on Raspberry Pi
- Click 'Save Impulse'

Step 3 — Train and Validate

- Go to 'Transfer Learning' tab and click 'Start Training'
- Training runs on Edge Impulse cloud — takes a few minutes

- Aim for at least 85% validation accuracy before deploying
- If accuracy is low: add more of your own webcam images and retrain
- Use the 'Model Testing' tab to verify on unseen test images

Step 4 — Download and Deploy

- Go to 'Deployment' tab → select 'Linux (AARCH64)' for Raspberry Pi 4
- Click 'Build' and download the .eim model file to your laptop
- Transfer the .eim file to your Raspberry Pi via USB drive or SCP over WiFi

6. Raspberry Pi Setup

Hardware Connections

- Connect USB webcam to any USB port on the Pi
- Connect Load Cell to HX711 → DATA to GPIO5, CLK to GPIO6
- Connect Servo Motor signal wire to GPIO18 (PWM-capable)
- Power servo separately — do NOT use Pi's 5V pin for the servo

Install Required Libraries

- pip3 install edge_impulse_linux — Edge Impulse Linux SDK
- pip3 install opencv-python — for USB webcam capture
- pip3 install RPi.GPIO — for servo motor control
- pip3 install hx711 — for weight sensor readings

USB Webcam Code

```
cap = cv2.VideoCapture(0) # 0 = first USB webcam. Try 1 or 2 if not detected.
```

7. Combined Scoring Logic

Final Score = (Edge Impulse Score × 0.40) + (Weight Score × 0.60)

If Final Score ≥ 0.5 → GOOD | If Final Score < 0.5 → BAD

Weight Score = (measured_weight - min_weight) / (max_weight - min_weight). Clamp between 0.0 and 1.0.

Scenario	EI Score	Weight Score	Final Score	Decision
Both agree (good)	0.90	0.80	0.84	GOOD
Both agree (bad)	0.20	0.30	0.26	BAD
Looks good, underweight	0.90	0.30	0.54	GOOD (marginal)
Looks bad, good weight	0.20	0.80	0.56	GOOD (marginal)

8. What TO Do

- ✓ Use Kaggle datasets to save time — search for 'fresh vs rotten fruit' datasets
- ✓ Always add 30-50 of your own webcam images to close the domain gap
- ✓ Open Edge Impulse on your laptop browser — not on the Raspberry Pi
- ✓ Calibrate the weight sensor at every system startup
- ✓ Start with 40/60 weightings and adjust based on real test results
- ✓ Log every sensor reading and final decision for model improvement
- ✓ Retrain the model if you switch to a different fruit type
- ✓ Test with at least 20-30 real fruit samples before going live
- ✓ Add a manual override button for edge cases or maintenance

9. What NOT To Do

- ✗ Do not use only Kaggle images — always mix in some of your own webcam photos
- ✗ Do not train the Edge Impulse model on the Raspberry Pi — use your laptop
- ✗ Do not skip model validation — check accuracy before deploying to the Pi
- ✗ Do not rely on one sensor alone — always use the combined scoring formula
- ✗ Do not power the servo from the Raspberry Pi 5V pin — use external power
- ✗ Do not change lighting after training without retraining the model
- ✗ Do not hard-code weight thresholds without real-world fruit testing
- ✗ Do not run the conveyor faster than the Pi can process each inference

10. Troubleshooting

Issue	Possible Cause	Fix
Low accuracy after Kaggle training	Domain gap — no own images	Add 30-50 own webcam images & retrain
Webcam not detected	Wrong device index	Try cv2.VideoCapture(1) or (2)
EI model always says BAD	Poor or unbalanced dataset	Balance classes; add own images
Weight reads 0 or garbage	HX711 wiring/calibration	Re-check wiring; re-calibrate
Servo not moving	Wrong GPIO or no PWM	Use GPIO18; verify PWM setup
Slow inference on Pi	Large model size	Switch to MobileNetV1 96x96
Can't find .eim file on Pi	File not transferred	Copy via USB drive or SCP